**Home** Content Contact

# INTRODUCTION

UNDERSTANDING THE BASICS

Start

# WHAT IS PROGRAMMING?

- A computer program is a sequence of instructions that directs a computer to perform certain actions in a specified order.
- Computer programs are typically written in a programming language, which is a language designed to facilitate the writing of instructions for computers.
- The act (and art) of writing a program is called programming.

# PROGRAMMING LANGUAGES

- Machine Language - Machine language is the lowest-level programming language made up of binary code (0s and 1s). It is the only language that a computer’s CPU can directly understand and execute.

- Low-Level Language - Low-level languages are closer to machine language and hardware. They use short, symbolic instructions (instead of pure binary) and give programmers more control over the computer’s hardware. Example: Assembly

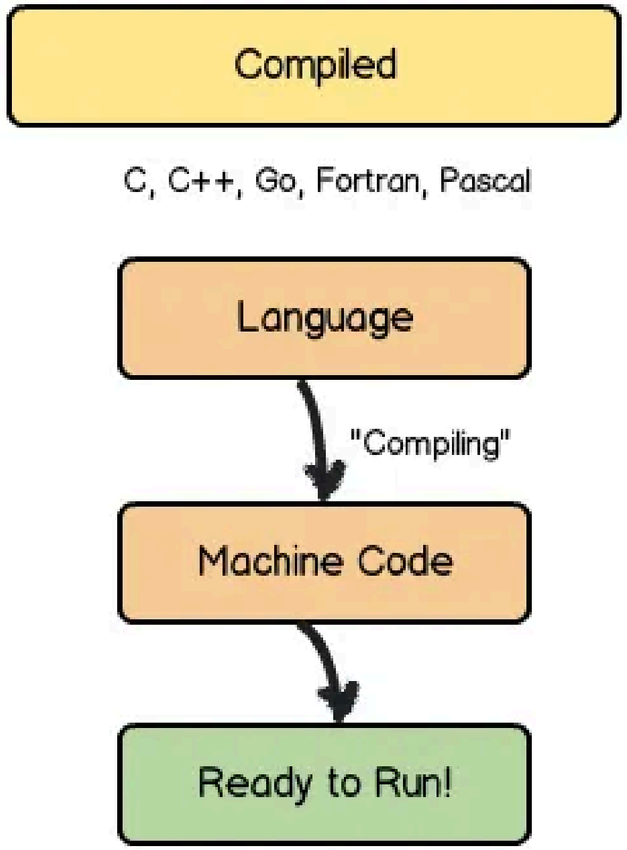
- High-Level Language - High-level languages are designed to be easy for humans to read, write, and understand. They use English-like words and symbols and are mostly independent of hardware.

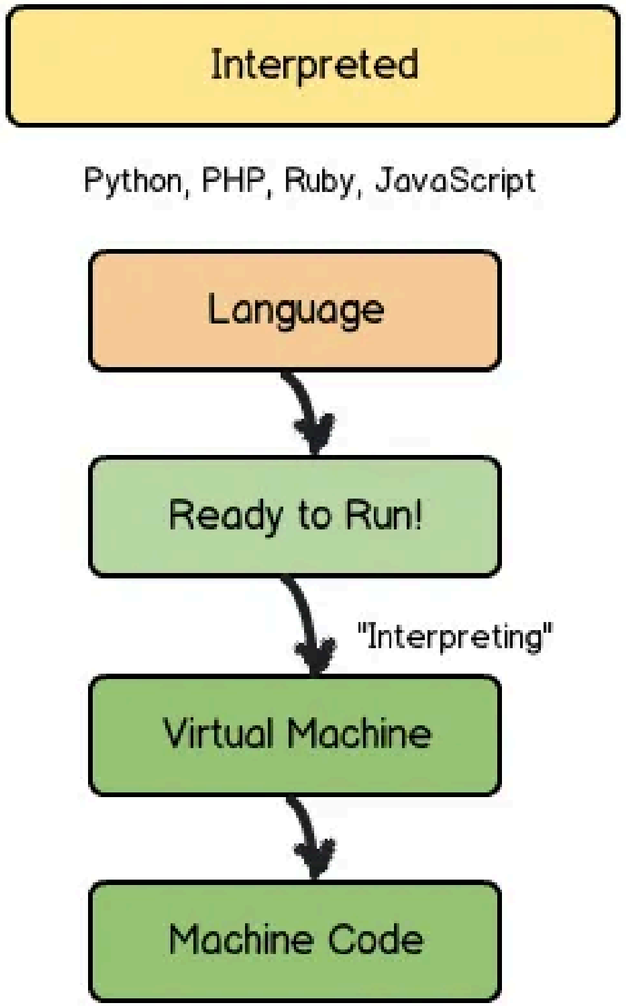
## COMPILED LANGUAGE

- Program is translated before it runs
- Produces an executable file
- Runs faster after compilation
- Errors are found before execution

## INTERPRETED LANGUAGE

- No separate executable file
- Code runs line by line
- Slower than compiled languages
- Errors appear during execution


Compiled
C, C++, Go, Fortran, Pascal
Language
"Compiling"
Machine Code
Ready to Run!


Interpreted
Python, PHP, Ruby, JavaScript
Language
Ready to Run!
"Interpreting"
Virtual Machine
Machine Code

# INTRODUCTION TO C/C++

## C++ Overview

- C++ is a high-level programming language
- Developed by Bjarne Stroustrup
- Extension of the C programming language

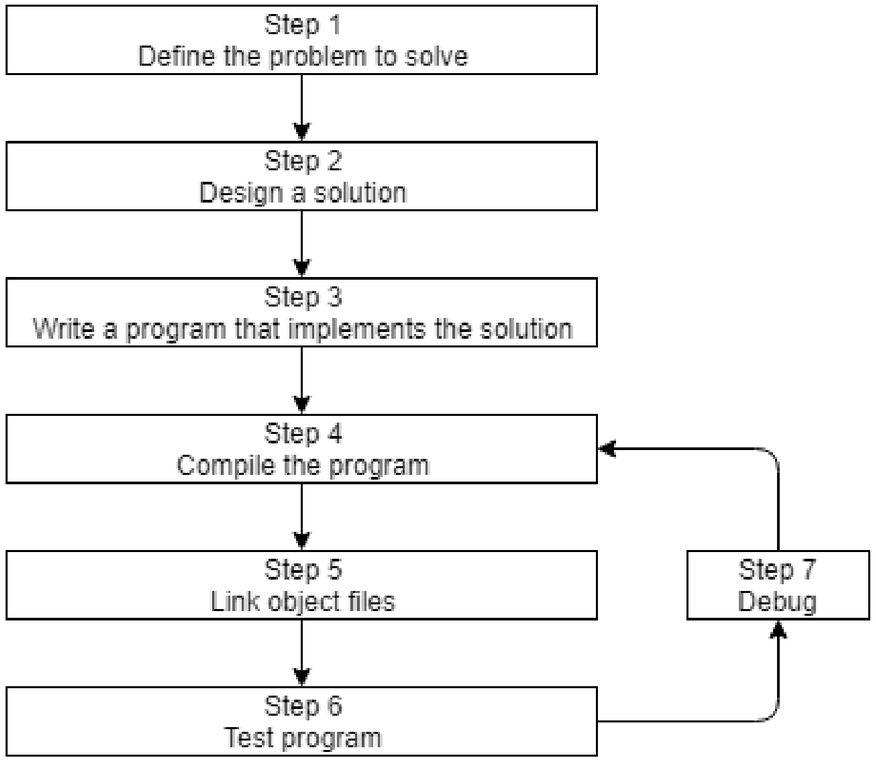
## Features of C++

- Compiled language (fast execution)
- Supports object-oriented programming
- Allows low-level memory control
- Efficient and powerful

## Why Use C++

- High performance
- Widely used in industry
- Good for learning core programming concepts

Before we can write and execute our first C++ program, we need to understand in more detail how C++ programs get developed. Here is a graphic outlining a simplistic approach:



# SETTING UP

- Since C++ is a compiled language, you need a compiler(gcc = C, g++ = c++)
- An IDE to write your program(Code:Blocks,VSCode)
- Adding it to your PATH
- Optional(VScode Extensions)

# INSTALLING AN INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)

AN INTEGRATED DEVELOPMENT ENVIRONMENT (IDE) IS A PIECE OF SOFTWARE DESIGNED TO MAKE IT EASY TO DEVELOP, BUILD, AND DEBUG YOUR PROGRAMS.

A TYPICAL MODERN IDE WILL INCLUDE:

- SOME WAY TO EASILY LOAD AND SAVE YOUR CODE FILES.
- A CODE EDITOR THAT HAS PROGRAMMING-FRIENDLY FEATURES, SUCH AS LINE NUMBERING, SYNTAX HIGHLIGHTING, INTEGRATED HELP, NAME COMPLETION, AND AUTOMATIC SOURCE CODE FORMATTING.
- A BASIC BUILD SYSTEM THAT WILL ALLOW YOU TO COMPILE AND LINK YOUR PROGRAM INTO AN EXECUTABLE, AND THEN RUN IT.
- AN INTEGRATED DEBUGGER TO MAKE IT EASIER TO FIND AND FIX SOFTWARE DEFECTS.
- SOME WAY TO INSTALL PLUGINS SO YOU CAN MODIFY THE IDE OR ADD CAPABILITIES SUCH AS VERSION CONTROL.

EXAMPLES: VISUAL STUDIO, CODE:BLOCKS, VSCODE

# WHAT IS THE PATH ENVIRONMENT VARIABLE?

- PATH is a system setting that tells Windows where to look for programs
- It contains a list of folders with executable files (.exe)

Example:

Adding C:\mingw64\bin to PATH allows g++ to run from any directory

Timmerman Industries

Home **Content** Contact

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;
}
```

← **Header**

← **Namespace Declaration**

← **Main Function**

← **Statement**

← **Return**

1. Header File: #include <iostream> adds input/output objects (cin, cout, etc.) via the preprocessor. Common headers: fstream (files), string (strings), vector (STL), bits/stdc++.h (all-in-one).
2. Namespace Declaration: using namespace std; allows direct use of standard names like "cout" without std::
3. Main Function: int main() is the program’s entry point; execution starts here and returns an integer with 0 return mean successful execution.
4. Comments: // for single line, /*...*/ for multi-line are ignored by the compiler and used only for code documentation.
5. Statement: Contains executable code. Here, cout << "Hello World!", prints the text on the screen using the insertion operator (<<).
6. Return: The return 0; statement terminates the main() function and indicates that the program executed successfully.

More Examples:

1.

```
#include <iostream>

int main() {
    std::cout << "This program uses a header file." << std::endl;
    return 0;
}
```

- #include <iostream> allows input/output
- Without this header, cout would cause a compile error

More Examples:

2.

```
#include <iostream>

using namespace std;

int main() {
    cout << "Namespace allows simpler names." << endl;
    return 0;
}
```

- using namespace std; avoids writing std:: repeatedly
- Demonstrates cleaner syntax for beginners

**ACTIVITY: “Think Like a Computer”**

1. **Choose one simple real-life task:**
- **Making Coffee or Noodles**
2. **Write a step-by-step instruction list as if:**
- **The reader is a computer**
- **The reader cannot guess or assume anything**
3. **Rules**
- **Stepss must be numbered**
- **No skipped steps**
4. **Answer the following questions:**
- **Which step in your instructions is the most important? Why?**
- **What would happen if the steps were done in the wrong order?**

Example Output:

“Buying Enrolling for the semester”

1. Bayad sa downpayment
2. Adto ccs faculty
3. pangitaon folder
4. Enlist portal
5. Give folder to sir badi
6. Enlist subjects depending on offer number
7. Submit
8. Wait for approval